

資料分析專案 · NGSIM 車輛軌跡

變換車道的 外部成本

一次換道，讓後方那台車付出多少代價？
誰付出得更多？

870 萬筆軌跡 · 2,472 次事件 · 開源可重現

假設與結果

我們假設貨車
傷害更大。
資料說相反。

而且控制路況之後，車種的影響與零沒有統計差別。

為什麼問這個問題

車隊管理系統評分駕駛，評的全是**對自己車輛的風險**。

沒有一項在評駕駛**對別人造成的代價**。那個代價駕駛看不到，因為它發生在他後面。

現有評分

急煞 · 超速 · 怠速 · 疲勞駕駛

沒有人正在評的

我的動作，讓後面的人付出多少？

資料：美國聯邦公路總署 NGSIM

架高攝影機逐格判讀真實高速公路車流，每秒 10 筆車輛位置。

路段	地點	原始筆數	去重後
I-80	加州 Emeryville	4,566,387	4,566,387
US-101	洛杉磯	4,802,933	4,098,933

US-101 有 **704,000 筆逐位元完全重複**的資料列，佔 14.7%。留著會讓那些車在每個平均值裡被算兩次。

排除 Lankershim 與 Peachtree：有紅綠燈，後車減速是紅燈不是換道

01

方法

怎麼從 870 萬筆軌跡，變成一個站得住的數字。

怎麼算一次「換道」

穩定性：前後各滿 3 秒

擋掉偵測跳動與連續蛇行。

必須有後車

沒有人承受代價，就沒有東西可量。

只留主線車道

匝道上的「換道」是被迫匯入，成因不同。

過濾漏斗

原始換道 4,031

穩定性 2,955

有後車 2,914

資料完整 2,531

對照組決定結論算不算數

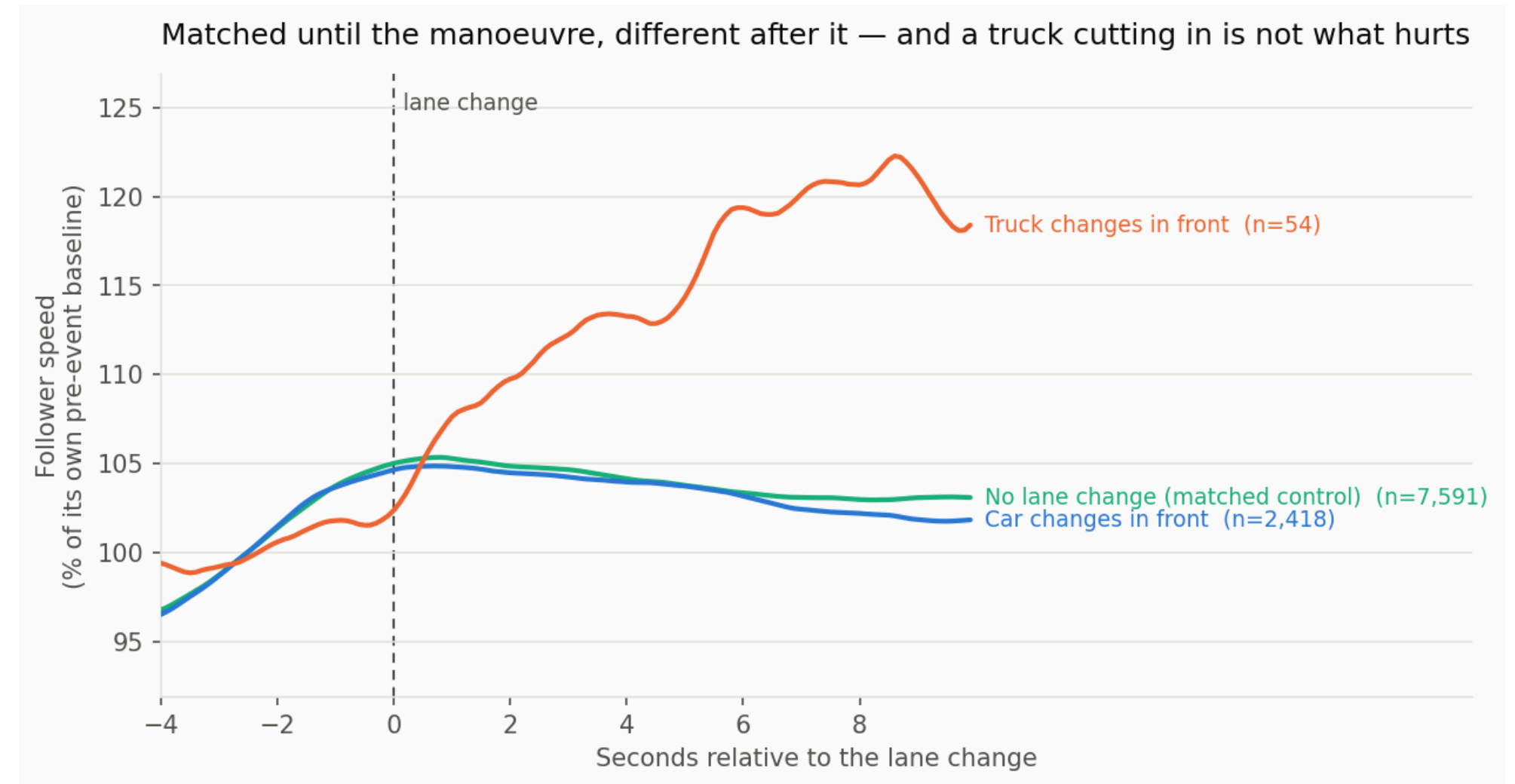
「換道後掉了 3.9 km/h」什麼都沒證明——塞車車流本來就在不停忽快忽慢。

所以每筆事件都對照**沒有換道、但條件相同的**時刻，用最近鄰配對三個**事前**變數：

原本開多快

原本在加速還是減速

前方原本有多少空間



兩條曲線在換道前**完全重疊**，之後才分開。這就是配對有效的視覺證據。

配對的三個變數，全部在事件發生前量測

```
# 最近鄰配對，不重複取樣，在同路段 × 同壅塞程度之內
cols = ["baseline_kmh", "pre_trend_kmh", "headway_pre_ft"]
mu, sd = c_raw.mean(axis=0), c_raw.std(axis=0)
tree = cKDTree((c_raw - mu) / sd)
_, nn = tree.query((e_raw - mu) / sd, k=k)
# headway_at_t 刻意不放進來：它是 treatment 之後的變數
# 被切入的定義就是車距變小，配它等於把機制配掉
```

第一版用了「換道當下的車距」。那是事後變數，**它把結論的正負號翻了過來**：-0.35 變成 +0.76。

用估計式沒看過的欄位驗證配對

車距沒有進入配對，所以它是獨立的檢查，不是循環論證。

	事前車距	換道當下	變化
換道事件	106.3 ft	63.7 ft	-42.6 ft
配對對照組	105.8 ft	102.7 ft	-3.2 ft

這證明兩件事：我們抓到的真的是**切入**，而對照組真的**沒被干擾**。

事前三個配對變數的平均值：30.86 vs 30.90、1.62 vs 1.61、106.3 vs 105.8

02

結果

2,472 次事件，7,591 個配對對照組。

換道確實有代價

後車多掉的速度

+0.76 km/h

95% 信賴區間 +0.44 ~ +1.09

恢復時間延遲

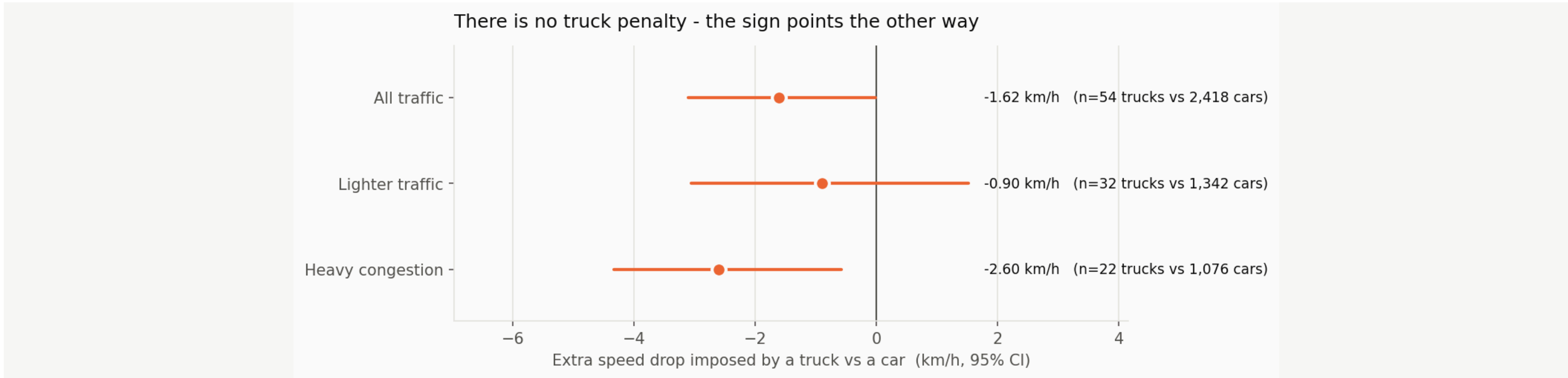
+0.27 秒

95% 信賴區間 +0.15 ~ +0.40

不同路況下後車速度損失長條圖，含信賴區間



但貨車的代價更低，不是更高



貨車切入讓後車少掉 **1.62 km/h**，恢復也快 **0.98 秒**。

而且貨車換道次數本來就比較少：每車小時 7.7 次 vs 10.7 次。

貨車事件僅 54 筆，對小客車 2,418 筆——所以全部報信賴區間，不報點估計

控制之後，撐下來的是路況

變數	係數	p 值	
是否為貨車	-1.364	0.468	不顯著
周圍車流平均速度	-0.351	9e-67	高度顯著
後車原本速度	+0.470	1e-151	顯著
車距	-0.035	3e-24	顯著

如果機制是「貨車又大又重」，車種係數應該撐得過控制變數。**它沒有**。撐過去的是路況：周圍車流每慢 1 km/h，同一動作多造成 0.35 km/h 的擾動。

核心結論

外部成本屬於 「時機」， 不屬於「車種」。

針對車種的管制瞄錯了變數。針對「什麼時候、切進多大的縫」才是對的。

我們也報了對自己不利的數字

凹陷深度

+0.76 km/h

顯著

恢復時間

+0.27 秒

顯著

後車淨損失時間

+0.03 秒

信賴區間跨過 0

擾動是**尖銳而短暫**的，不是持續的損失。我們的分析**不支持**「一次換道讓後車損失可測量的行車時間」，我們也沒有這樣寫。

擾動會不會向上游放大成連鎖壅塞，這份資料的路段只有 500–640 公尺，答不了

四個 bug，三個在摘要統計上看不出來

	問題	怎麼被發現的
D5	兩段錄影時間連續，切不開，88 個車號撞號	不相干的當機
D8	第一版對照組在整個視窗內加速 23%	把曲線畫出來
D9	官方資料有 704,000 筆完全重複的資料列	不相干的當機
D10	配對用了事後變數，結論正負號翻掉	想清楚因果順序

每一個決策與每一個錯誤都寫進公開的 decision log，共 11 條。

tests/test_detection.py

22 項合成軌跡檢驗

```
$ python tests/test_detection.py
PASS 單格跳動被穩定性過濾擋掉
PASS 連續蛇行被擋掉
PASS 注入的減速幅度量得出來
PASS 不恢復的情況正確標記為截斷
PASS 時間上連續的兩段錄影仍然被切開
all checks passed
```

用手工造的軌跡測核心邏輯——答案是我們自己造的，所以已知。

藍色那一項是 D5 那個 bug 的**回歸測試**：確保它不會再發生。

每次 run_all.sh 都會先跑測試，不過就不繼續

可重現性

重跑一次， 每個數字都一樣。

連 bootstrap 信賴區間到小數點後兩位都相同——固定亂數種子。

```
$ bash scripts/run_all.sh
```

下載 → 偵測事件 → 建對照組 → 統計 → 圖表 → 字數檢查